

CS3219 AY2627S1 Project Description

Expected Learning outcomes

- You should be able to apply the concepts learned in the lectures in a real-world context by designing a useful application that can be added to your portfolios.
- You should be able to extend your existing software engineering knowledge by:
 - developing a requirements specification (containing both functional and non-functional requirements),
 - developing an architectural design specification that satisfies the requirement specification, and
 - designing and developing an application following relevant design principles and patterns.
- You should be able to explore various architectures and design decisions in implementing the features of the product.
- You should be able to improve your communication skills (technical and non-technical) through collaborative group work, technical documentation writing, and presentations.

Project context

In this project, you will develop a medium-scale web application. The project involves designing and developing a peer-to-peer campus errand platform where students can request items to be collected from stores or facilities within the campus, and other students can fulfill (and deliver) these requests. The platform operates using a closed credit economy (i.e., credits cannot be purchased, withdrawn, or exchanged for money, and only circulate within the platform). During sign-up, each student receives an initial allocation of X credits (teams can assign any reasonable value to X) and earns additional credits by completing errands for other students.

A general description of the use of the platform is as follows:

*A student who needs one or more items collected acts as a **requester**. The requester creates an account, logs in, and submits an errand request by selecting a pickup location and specifying the required delivery details. The system reserves one or more available credits when the request is created. Other students may act as **couriers** by viewing open requests and accepting one. Once a request is accepted, the assigned courier collects the items and updates the same in the system. After the items are delivered and the delivery is confirmed, the reserved credits are transferred from the requester to the courier. If the request is canceled or is not accepted within*

the permitted duration, it should expire or be canceled appropriately, and the reserved credits should be released.

Students may use the platform both to request errands and to fulfill requests submitted by others. You are encouraged to explore the requirements and architectural design of such an application without deviating from its overarching purpose.

Implementation Expectations

All mandatory features/functionality, behavior, and deployment related descriptions are labelled as Mx in the following.

M1. Responsive Design and User Experience

The web application should provide a responsive, intuitive user interface that works across desktop and mobile screen sizes and supports complete workflows for both requesters and couriers. You are encouraged to demonstrate strong product thinking in screen organization, navigation, user feedback, and handling invalid or unsuccessful actions.

2. Mandatory Microservices

The application should follow a microservice-based architecture and include the following mandatory services.

M2. User Service

The User Service should manage user registration, authentication, profile information, and a user's ability to participate as both a requester and a courier.

M3. Supplier Service

The Supplier Service should maintain information about the stores (e.g., CoffeeBean@Com3), facilities (e.g., Printers@PCCommons), and/or locations (e.g., PGP Foyer) from which users may request errands. It should support CRUD on campus suppliers/vendors/landmarks. Use the initial data provided in the template repository to seed your application. You are expected to add more to ensure a rich listing.

M4. Order Service

The Order Service should manage the complete lifecycle of an errand request. This includes creating a request, listing open requests, accepting a request, courier assignment, marking an item/request as picked up, completing or canceling a request, and handling requests that expire before being accepted. You are expected to handle corner cases such as ensuring only one courier can be assigned to an order and that the requester cannot accept their own request.

M5. Credit Service

The Credit Service manages the closed credit economy of the platform. Users receive an initial credit allocation when they register. When an errand request is created, the system should reserve the required credits from the requester's available balance and transfer them to the courier upon successful completion of the errand. Credits should be released if the request is canceled or expires.

Note: You are expected to add additional services where appropriate.

M6. Asynchronous/Event-Driven Communication

The application must include meaningful asynchronous or event-driven (or both) workflows. You should identify interactions that do not require immediate synchronous communication and are suitable for asynchronous handling. The selected workflow(s) should support a genuine product or system requirement.

M7. Containerization

All application services and supporting components should be containerized (e.g., using Docker) and deployed locally (i.e., on your computers) or on the cloud. Note: While deployment on cloud is strongly encouraged, the scope of must-have deployment is limited to local deployment.

Assumptions and Scope Boundaries

The following assumptions may be made while designing the application:

- Suppliers and valid campus locations may be predefined in the system.
- Payment for the items collected from a supplier is outside the scope of the platform.
- Platform credits represent compensation for completing an errand and are not used to pay the vendor.
- Supplier menus, product catalogs, inventory, and checkout are outside the project scope.
- Courier (live) location-tracking during transit is outside the project scope.

Students may make additional reasonable assumptions where requirements are not explicitly specified. Such assumptions should be documented and should not alter the overarching purpose of the application.

Note: When in doubt, get your assumptions clarified by talking to your mentor or the course coordinator.

Nice-to-Have Features

The following are the nice-to-have (N2H) categories, labelled Nx, identified by the teaching team for this project. They are neither exhaustive nor prescriptive, and your team is free to design, adapt, or extend them based on your interpretation of the project.

You are not required to limit yourselves strictly to the categories or items listed below. If your team identifies features that are more aligned with the objectives of the project, you may propose and implement them instead. In such cases, discuss the proposed N2H features with your mentor and lecturer before implementation to ensure that they remain within the scope and purpose of the project, and are of appropriate complexity.

While there is no fixed limit on the number of N2H features a team may implement, every team member must contribute to the implementation of at least one N2H feature worth of effort. Teams should distribute the work reasonably equally among team members and ensure that the overall workload remains manageable within the project timeline.

When presenting an N2H feature, your team should explain its relevance to the platform, the problem it addresses, and the technical design and implementation choices involved. The objective is not simply to implement a larger number of features, but to use the selected N2Hs to demonstrate your team's strengths, product thinking, and ability to independently explore relevant technologies and design approaches.

Some of the concepts required to implement the must-have or N2H features may not be explicitly covered in lectures or tutorials. Teams are therefore encouraged to conduct independent research where required.

N1: Application Enhancements

These features extend the functionality and user experience of the core application. Possible enhancements are as follows:

- **Requester enhancements**
 - Scheduled errands
 - Order history + Reordering errand flow
 - Bookmarking delivery and pickup locations
- **Courier enhancements**
 - Route visualization and navigation support
- **Communication**
 - Real-time chat, voice/video call
 - Real-time status update push notifications to user
- **Admin Dashboard**
 - Live and past orders listing
 - Advanced user management flows, e.g., user suspension, account review and reinstatement based on evidence drawn from system logs
 - Failed-order and dispute management
 - User/courier login audit logs
 - Application-level, user-level, task-level analytics

N2: AI Features and External Integrations

These features introduce intelligent capabilities or integrate the application with external systems and services. Possible enhancements include:

- **Natural-language or voice-based errand creation:** Allow requesters to describe an errand using natural language or voice, with the system extracting the relevant details and converting them into a structured errand request.
- **Real-time chat translation:** Translate messages between requesters and couriers in real time to support communication across the multilingual NUS student community.
- **AI-assisted proof-of-delivery verification:** Create a proof of delivery flow where AI analyses the images submitted by the courier to verify whether an errand appears to have been successfully completed.
- **RAG-based FAQ/support:** A RAG-based chatbot to help with FAQ and support (with reasonable supporting documentation created, system logging, etc) to answer user's questions.
- **Task-credits-estimation assistant:** Analytics based task-credits estimation assistant to recommend credits allocation during new tasks creation, or verification of credit assignment when accepting an errand.
- **Content moderation on in-app communication:** Flag/mute abusive, unsafe communication between requester and courier before being delivered.
- **Smart request-to-courier ranking/matching:** Use an LLM/ranking model to match requests and explain to the courier why the request is a good match.

N3: Testing, Reliability, and Observability

These features improve confidence in the correctness, reliability, and operational behavior of the application. Possible enhancements include:

- comprehensive automated testing beyond the unit- and integration-testing, including end-to-end testing, load, stress, or concurrency testing, demonstrated with appropriate evidence
- centralized logging
- automated health monitoring and alerts via monitoring dashboards
- distributed message/communication tracing
- enhanced handling and recovery of failed asynchronous messages
- responsible handling of data sent to third-party services and LLMs

N4: Cloud deployment and DevOps

These features extend the mandatory containerized setup and demonstrate more advanced deployment and software delivery practices. Possible enhancements include:

- establishing CI/CD pipelines
- deployment to a cloud platform
- Kubernetes deployment and automated scaling, demonstrated with evidence
- automated build, test, and deployment workflows
- infrastructure-as-code; secrets and configuration management
- production-oriented deployment and monitoring setup

Organizing work within your Project Team

- Each project team will have 4 to 5 members. Teams of less than four or more than five members are disallowed.
 - For better collaboration, it is strongly encouraged to form teams within the same tutorial.
- Allocate the responsibility of completing the product features (i.e., must-haves and nice-to-haves) and the remaining project tasks (i.e., documentation, demo, and presentations preparation) roughly equally among the team members.
- Keep a note of the individual contributions. This will be factored in while grading the project components and marks for individual members may differ.
 - You are required to declare your role allocation and responsibilities in the final presentation.
 - Additionally, we will use peer reviews and mentors' feedback to gather information about the individual contributions and team dynamics. The teaching team will intervene if necessary.
- A mentor will be allocated to each project team.

Reuse policy and declaration

- Each group is expected to sign a declaration (see [Appendix 1](#)) that the project is bona fide work of the team.
- If any code is reused, or any reference is made to an existing design or documentation, acknowledge the source(s).
- Attach this declaration to your final submission (the presentation slides) before submitting it on Canvas.
- You are expected to read through the course's AI usage policy in [Appendix 2](#).

Project Code Management

You will be given a template repository, which **should be forked** to your team organization.

- The team organization name should be named as follows: AY2627S1-CS3219-P<groupnumber> (e.g., AY2627S1-CS3219-P1, AY2627S1-CS3219-P11, etc.,).

- You should use a single-repository structure with one folder per microservice.
- You should create a PR to the template repository from your team organization's repository to facilitate code review and contribution tracking, where necessary.

Project Timeline

To facilitate incremental work on the project, we have 4 graded milestones during the semester. A summary is provided below.

Note: All graded milestones are in-person/face-to-face assessments. All team members are expected to be present irrespective of who contributed to the deliverable.

Milestones	Deliverables	Weightage (%)	Submission Format and Deadline
D1	Project backlog and prototype presentation (defining project scope)	5	1/ F2F presentation (20 min) to the lecturer + TA/mentor, followed by QA (10 min) before the end of Week 5 (Between Sep 9-12, 2026); 2/ Backlog document submission on Canvas by Sep 17, 2026 at 23:59
D2	First graded progress check	5	1/ F2F presentation (including demonstration) to mentor in Week 7.
D3	Second graded progress check	10	1/ F2F presentation (including demonstration) to mentor in Week 10
D4	Full project demonstration with slides.	20	1/ Slides to be submitted and product final commit by Nov 11, 2026, 10:00 hrs. 2/ F2F presentation with demo (30 min), followed by QA (10 min) to graders before the end of Week 13 (between Nov 11- 14, 2026). Demo slot selection will open at least a week before.